

PlayerPrefsRuntime Tool

version 3.1

Overview

PlayerPrefsRuntime Tool is a Unity plugin that allows you to retrieve and edit all PlayerPrefs at runtime across multiple platforms, including Android, iOS, Windows, macOS and WebGL (with Windows/macOS also supported in the Unity Editor). It provides a public mutation API, export/import, an editor window that works without Play Mode, and an interactive UI viewer. This tool is invaluable for debugging, analytics, testing, and managing player preferences within your Unity projects.

Dependencies

PlayerPrefsRuntime Tool uses a JSON parser: `com.unity.nuget.newtonsoft-json` (pinned at 3.2.1). If it is missing, add it through the Package Manager.

Features

- **Cross-Platform Support:** Compatible with Android, iOS, Windows, macOS and WebGL (plus Windows/macOS Editor).
- **Runtime Access:** Retrieve all PlayerPrefs as a dictionary directly from the device (or editor storage on Windows/macOS).
- **Live Editing:** Edit, save, add and delete PlayerPrefs entries directly from the runtime UI.
- **Export / Import:** Export all PlayerPrefs to a typed JSON document (clipboard + file) and import them back, with automatic backups before destructive operations.
- **Public API & Events:** `SetInt/SetFloat/SetString/DeleteKey/DeleteAll/ExportToJson/ImportFromJson` and an `OnEntryChanged` event.
- **Editor Window:** View/edit/export/import PlayerPrefs from the Tools menu without entering Play Mode.
- **Logging:** Display all PlayerPrefs data in the Unity Console for debugging purposes.
- **Interactive UI Viewer:** Visual interface with search, type filters, sorting, size indicators, detailed entry inspection and edit mode.
- **Type Validation:** Automatic validation for Int, Float and String values with error feedback.
- **Extensible Architecture:** Easily extendable to support additional platforms or functionality.

UI Viewer Features

The PlayerPrefsRuntime Tool includes a comprehensive visual UI viewer that provides:

Real-time Search & Filtering

- Live search across key names, types and values

- Case-insensitive filtering with instant results
- Type filter chips (Int / Float / String)
- Clear button to reset search; shows filtered count vs total count in the header

Sorting Capabilities

- Toggle between "Sort: Name" and "Sort: Type" modes
- Secondary sorting (type within name, or name within type)
- Alphabetical ordering using ordinal comparison

Visual Design & Theming

- Dark theme with accent colors (blues, cyans)
- Alternating even/odd row colors for readability
- Color-coded type badges (Int, String, Float)
- Per-entry and total data size indicators
- Safe area support for mobile devices
- Virtualized (recycled-row) list for thousands of entries

Interactive Features

- Click any entry to view detailed information in a dialog (with JSON pretty-print and Copy Value)
- Toggle edit mode with the "Edit" button; modify Int, Float and String values directly
- Save changes with the "Save" button; delete entries with confirmation
- Add new entries (key + type selector + overwrite guard); Delete All with confirmation
- Export / Import via the header action bar; automatic UI refresh after any change
- Opt-in device gesture (hold three fingers for two seconds) to open the viewer

Notes & Limitations

- The tool is compiled conditionally behind the `PLAYER_PREFS_RUNTIME_TOOL` define constraint (see `PlayerPrefsRuntime.asmdef`). If the define is not set, the API will not exist in your project.
- Windows reads `PlayerPrefs` from the registry (`HKCU/Software/{company}/{product}` in player, and `HKCU/Software/Unity/UnityEditor/{company}/{product}` in editor) and strips Unity's hashed suffix (e.g. `_h123`). Returned values are best-effort decoded and may include `int`, `long`, `float`, `string` or `byte[]`.
- WebGL parses Unity's binary `PlayerPrefs` (UPP) file from the Emscripten `/idbfs` mount (no JavaScript plugin); a 1 MB total size cap is surfaced in the viewer.
- Android reads `SharedPreferences` from `#{Application.identifier}.v2.playerprefs` and supports `int`, `float` and `string` (other types are stringified).

- iOS exports only NSString/NSNumber values from UserDefaults. macOS runtime reads ~/Library/Preferences/*.plist; non-JSON plist types can cause an empty result. macOS Editor parsing supports XML and binary plists (via plutil).

Platform Support Details

Platform	Implementation	UI Viewer	Method
Android	PlayerPrefsRuntimeFetcherAndroid — SharedPreferences via AndroidJavaObject	Yes	Runtime
iOS	PlayerPrefsRuntimeFetcherIOS — native P/Invoke to UserDefaults	Yes	Runtime
macOS	PlayerPrefsRuntimeFetcherMacOS — native P/Invoke to UserDefaults	Yes	Runtime
Windows	PlayerPrefsRuntimeFetcherWindows — registry access with binary parsing	Yes	Runtime
WebGL	PlayerPrefsRuntimeFetcherWebGL — binary (UPP) parser reading the IDBFS mount	Yes	Runtime
Editor (Win)	PlayerPrefsRuntimeFetcherWindowsEditor — registry access	Yes	Editor only
Editor (Mac)	PlayerPrefsRuntimeFetcherMacOSEditor — plist file parsing (XML/binary)	Yes	Editor only

Platform-Specific Features

Android

- Accesses SharedPreferences using the package name
- Handles URI-encoded keys and values
- Supports standard int, float, string types

iOS / macOS

- Native JSON export via P/Invoke (PlayerPrefsRuntimePlugin.mm)
- Native plugins located in Plugins/iOS/ and Plugins/macOS/
- iOS exports only strings/numbers from UserDefaults; macOS runtime depends on plist values being JSON-serializable

Windows

- Registry-based access at `Software/{company}/{product}`
- Custom binary parsing for Unity's PlayerPrefs format
- Best-effort decoding: `int`, `long`, `float`, `string` and sometimes `byte[]`

WebGL

- Parses Unity's binary PlayerPrefs (UPP) file from the Emscripten `/idbfs` mount — no JS plugin
- 1 MB total size cap surfaced in the viewer

Data Type Handling

Automatic normalization across platforms via `PlayerPrefsRuntimeJsonHelper`. Type consistency ensures `long`→`int` and `double`→`float` conversions. Null safety provides graceful handling of missing or corrupted data.

What's New in Version 3.1

- **Add / Delete All / Export / Import** in the runtime viewer, with confirmation dialogs and automatic JSON backups before destructive operations.
- **Public mutation API** (`SetInt/SetFloat/SetString/DeleteKey/DeleteAll/ExportToJson/ImportFromJson`), the **OnEntryChanged** event and the **Version** constant.
- **Editor Window** (Tools > PlayerPrefs Runtime Viewer) — view/edit/export/import without Play Mode.
- **WebGL support** via a binary PlayerPrefs (UPP) parser reading the Emscripten IDBFS mount.
- **Device gesture trigger** (opt-in), **type filter chips**, **data size indicators**, JSON pretty-print and Copy Value.
- **Virtualized** entry list for thousands of entries; large values are safely capped in the detail dialog so the legacy UI Text renders them.
- **Centralized writer**: all writes (runtime UI, editor window, public API) go through one mutation core.
- **Unit tests**: EditMode suite (normalization, serializer, registry decoder, WebGL parser, entry model, value display, writer) plus PlayMode viewer smoke tests. See TESTING.md.

Previously in Version 3.0

Version 3.0 introduced runtime Edit Mode: toggling edit mode in the detail dialog, type-safe editing of `Int/Float/String` values, saving directly to PlayerPrefs, deleting entries and automatic UI refresh.

Use Cases

- **Testing**: Quickly modify values to test different game states
- **Debugging**: Fix corrupted or incorrect PlayerPrefs data on the fly
- **QA**: Adjust test parameters without rebuilding the app
- **Development**: Iterate faster by changing values in real-time

Installation

1. Get the Plugin

- Option A (recommended): install from the Unity Asset Store — PlayerPrefs Runtime Tool with Viewer.
- Option B: import the Unity package `PlayerPrefsRuntimeTool.unitypackage` via Assets > Import Package > Custom Package.
- Option C: clone the repository and copy `Assets/DmytroUdovychenko/PlayerPrefsRuntimeTool` into your Unity project.

2. Enable the Tool

Define the `PLAYER_PREFS_RUNTIME_TOOL` scripting symbol: Edit > Project Settings > Player > Other Settings > Scripting Define Symbols, then add `PLAYER_PREFS_RUNTIME_TOOL` (semicolon-separated if other symbols are present).

3. UI Viewer Setup (Optional)

The UI Viewer is automatically configured and requires no additional setup. Customize its appearance by editing `Scripts/PlayerPrefsView/PlayerPrefsRuntimeViewConstants.cs` (panel/row/text colors, font sizes, text limits, scroll and canvas-scaling settings). The viewer automatically creates the required components (Event System, Canvas) if they do not exist.

Usage

Basic API Usage

```
using System.Collections.Generic;
using UnityEngine;
using DmytroUdovychenko.PlayerPrefsRuntimeTool;

public class PlayerPrefsRuntimeExample : MonoBehaviour
{
    private void Start()
    {
        #if PLAYER_PREFS_RUNTIME_TOOL
            // Retrieve all PlayerPrefs as Dictionary<key, value>
            Dictionary<string, object> allPrefs = PlayerPrefsRuntime.GetAllPlayerPrefs();

            // Log all PlayerPrefs to the console
            PlayerPrefsRuntime.LogAllPlayerPrefs();

            // Show the interactive UI viewer
            PlayerPrefsRuntime.ShowAllPlayerPrefs();
        #endif
    }
}
```

Editing, export/import and events

```

#if PLAYER_PREFS_RUNTIME_TOOL
PlayerPrefsRuntime.SetInt("player_score", 42);
PlayerPrefsRuntime.SetString("player_name", "Alice");
PlayerPrefsRuntime.DeleteKey("obsolete_flag");

string json = PlayerPrefsRuntime.ExportToJson();
PlayerPrefsRuntime.ImportResult result = PlayerPrefsRuntime.ImportFromJson(json);

PlayerPrefsRuntime.OnEntryChanged += key =>
    Debug.Log($"Changed: {key ?? "(all cleared)"}");
#endif

```

Testing

The package ships with an EditMode unit-test suite (assembly `PlayerPrefsRuntime.Tests`) covering the pure logic: JSON normalization, the export/import serializer, the Windows registry decoder, the WebGL UPP parser, the entry value model, the entry-dialog value display and the mutation core (writer). PlayMode smoke tests exercise the viewer lifecycle. Run them from Window > General > Test Runner, or headless with the `-runTests` command-line flag. See `TESTING.md` for details.

Demo Scene

The included demo scene

(`Assets/DmytroUdovychenko/PlayerPrefsRuntimeTool/Demo/DemoScene.unity`) showcases diverse test data (short/long/Unicode strings, integers, floats, JSON, special characters), full Unicode support, large-dataset performance and cross-platform consistency. Press Play, then press "P" to toggle the viewer (close via the X button); demo buttons also cover Show Viewer / Log All / Reset Demo Data.

License

This project is licensed under the MIT License.

API Reference

Member	Description
<code>PlayerPrefsRuntime.GetAllPlayerPrefs()</code>	Returns Dictionary<string, object> with all PlayerPrefs
<code>PlayerPrefsRuntime.LogAllPlayerPrefs()</code>	Logs all entries to the Unity Console
<code>PlayerPrefsRuntime.ShowAllPlayerPrefs()</code>	Shows the interactive UI viewer
<code>PlayerPrefsRuntime.IsVisible</code>	Returns true if the UI viewer is currently visible

Member	Description
<code>PlayerPrefsRuntime.SetInt/SetFloat/SetString</code>	Writes a typed value and saves immediately
<code>PlayerPrefsRuntime.DeleteKey(key)</code>	Deletes a single key and saves
<code>PlayerPrefsRuntime.DeleteAll()</code>	Deletes all PlayerPrefs (writes a JSON backup first)
<code>PlayerPrefsRuntime.ExportToJson()</code>	Exports all PlayerPrefs to a typed JSON string
<code>PlayerPrefsRuntime.ImportFromJson(json)</code>	Imports entries from JSON; returns a per-key result
<code>PlayerPrefsRuntime.OnEntryChanged</code>	Event raised after any entry changes (null key = DeleteAll)
<code>PlayerPrefsRuntime.EnableGestureTrigger()</code>	Enables the opt-in 3-finger open gesture
<code>PlayerPrefsRuntime.Version</code>	Package version constant (e.g. "3.1.0")

Contact

For any questions, suggestions or feedback, please contact:

LinkedIn: <https://www.linkedin.com/in/dmytro-udovychenko/>